Government Knowledge, Rules & Document Assistant

Prototype-to-Production Project Roadmap

A phased implementation plan for a secure, citation-backed government RAG assistant using Next.js, NestJS, FastAPI, PostgreSQL and pgvector.

Core principle

Approved government knowledge → retrieval → grounded AI response → source citation → human verification.

1. Entire Project — What You Are Building

The prototype is a secure government knowledge assistant for authorised officials. It does not make government decisions autonomously. Instead, it helps officials find approved information, answer routine procedural questions, summarize documents, extract structured information, and prepare drafts while preserving traceability to the original source.

Two roles

• **Admin / Knowledge Manager:** manages approved knowledge. Can upload documents, review processing, approve/reject/archive documents, manage officers, and inspect audit logs.

• **Officer:** consumes approved knowledge. Can search, ask questions, inspect citations, summarize documents, extract information, and generate drafts.

High-level architecture

```
Next.js Frontend
↓
NestJS Main Backend — authentication, RBAC, users, documents, approvals, audit logs, chat API
↓ internal API
FastAPI AI Service — OCR/text extraction, chunking, embeddings, retrieval, reranking, LLM,
citations
↓
PostgreSQL + pgvector — application data, document chunks and vector embeddings
■ object/file storage — original PDFs
```

Two main pipelines

Ingestion: Admin uploads PDF → NestJS validates and stores metadata/file → FastAPI extracts text/OCR → page-aware chunking → embeddings → store chunks/vectors → Admin approves → document becomes searchable.

Question answering: Officer asks question → NestJS authenticates/authorizes and records query → FastAPI embeds query → searches approved chunks → optionally reranks → LLM generates answer only from evidence → citations returned → Officer opens source page → action is audited.

Core product loop

The product is not simply a chatbot. Its core value is **approved knowledge + retrieval + evidence + AI + access control + auditability + human verification**.

2. Prototype Scope and Boundaries

Build a small but reliable prototype. Do not attempt a full government-wide platform.

• Use non-sensitive or explicitly permitted sample government documents for the pilot.

• Keep exactly two roles initially: ADMIN and OFFICER.

• Only Admin can upload/publish knowledge-base documents.

• Only APPROVED documents are eligible for officer-facing RAG retrieval.

• The AI must refuse or clearly state that evidence is insufficient when the approved knowledge base does not support an answer.

• Keep page numbers, document IDs, versions and chunk IDs throughout ingestion so citations can be verified.

Recommended first demo

• Admin logs in and uploads an approved sample PDF.

• System processes it and shows its status.

• Admin approves it.

• Officer asks a procedural question.

• Assistant answers with page-level citations.

• Officer clicks a citation and sees the original page/snippet.

• Officer requests a draft or summary.

• Admin opens the audit log and sees the activity.

3. Phase 1 — Project Foundation

Goal: Create the repositories, environments and service boundaries before implementing features.

Build:

• Create Next.js frontend.

• Create NestJS backend.

• Create FastAPI AI service.

• Set up PostgreSQL and enable pgvector.

• Choose file/object storage for PDFs; use local storage during development if needed.

• Set environment variables and separate development configuration.

• Define API conventions and error handling.

• Set up Git repository and a clean README.

Phase complete when:

• Frontend, NestJS and FastAPI all run independently.

• NestJS can connect to PostgreSQL.

• FastAPI can connect to the same PostgreSQL instance.

• A simple health-check works for each service.

4. Phase 2 — Authentication and RBAC

Goal: Create the security boundary around the application.

Build:

• Create User model with name, email, password hash, role, department and status.

• Implement login/logout/me endpoints in NestJS.

• Use JWT with an httpOnly cookie or another secure session approach.

• Implement authentication guard.

• Implement role-based authorization.

• Create ADMIN and OFFICER roles.

• Protect upload, approval, user-management and audit endpoints.

• Protect officer features behind authentication.

Phase complete when:

• An unauthenticated request cannot access protected APIs.

• An Officer receives 403 for Admin-only operations.

• Admin and Officer see appropriate dashboards and navigation.

• Authorization is enforced in NestJS, not only by hiding frontend buttons.

5. Phase 3 — Document Management

Goal: Create the authoritative document layer owned by NestJS.

Build:

• Create document model: title, department, document type, date, document number, version, status, file URL, uploader, timestamps.

• Implement PDF upload for Admin.

• Validate file type and size.

• Store original PDF in file/object storage.

• Store metadata in PostgreSQL.

• Implement document statuses: PROCESSING, PENDING_APPROVAL, APPROVED, REJECTED, SUPERSEDED.

• Create Admin document list, details and approval screens.

Phase complete when:

• Admin can upload a PDF.

• The original file is retrievable.

• Metadata and status are stored correctly.

• Officers cannot upload documents.

• Only approved documents are intended to enter the officer knowledge base.

6. Phase 4 — AI Document Ingestion Pipeline

Goal: Build the core RAG ingestion logic in FastAPI.

Build:

• Receive a document-processing request from NestJS.

• Extract text from normal PDFs.

• Add OCR fallback for scanned PDFs.

• Preserve page boundaries.

• Clean extracted text without destroying meaningful structure.

• Chunk text using a sensible chunk size and overlap.

• Attach metadata to every chunk: document ID, page number, section if available, version, department and status.

• Generate embeddings.

• Store chunks and embeddings in PostgreSQL + pgvector.

• Return processing status to NestJS.

Phase complete when:

• A sample PDF becomes searchable chunks.

• Every chunk retains its source page.

• Embeddings are stored successfully.

• Processing errors are visible instead of silently failing.

7. Phase 5 — Retrieval and RAG Question Answering

Goal: Turn the document store into a grounded assistant.

Build:

• Implement query embedding.

• Search pgvector for semantically relevant chunks.

• Filter retrieval to approved documents.

• Pass the top evidence passages to the LLM.

• Use a strict grounding prompt: answer from supplied evidence; do not invent unsupported facts.

• Return answer text plus source metadata.

• Store query and response metadata for auditing.

• Add a clear insufficient-evidence response.

Phase complete when:

• A question retrieves the expected source passages.

• The answer is grounded in retrieved evidence.

• The response contains document and page citations.

• Questions unsupported by the knowledge base do not receive fabricated answers.

8. Phase 6 — Citation and Source Verification

Goal: Make traceability the centerpiece of the prototype.

Build:

• Return document ID, title, page number, chunk ID and relevant source text with each citation.

• Build a source panel in the frontend.

• Allow the user to open the original PDF.

• Navigate directly to the cited page.

• Show the relevant source passage where practical.

• Clearly separate generated text from source evidence.

Phase complete when:

• An officer can go from an AI claim to the original source in a few clicks.

• Page numbers are consistent between ingestion and the viewer.

• The demo can prove where an answer came from.

9. Phase 7 — Search, Summarization and Extraction

Goal: Expand the assistant beyond conversational Q&A; while reusing the same document infrastructure.

Build:

• Build normal document search with filters.

• Add document summarization.

• Add structured extraction for fields such as document number, dates, department, applicability and key provisions.

• Keep citations attached to extracted facts where possible.

• Allow an officer to select a document and choose the desired operation.

Phase complete when:

• Officer can find documents without asking the LLM.

• A document can be summarized.

• Important fields can be extracted into structured output.

• Generated summaries/extractions remain traceable.

10. Phase 8 — Drafting Assistant

Goal: Support the drafting workflow described in the challenge without making the AI an autonomous decision-maker.

Build:

• Add a draft-note or draft-order workflow.

• Let the officer provide context and select relevant sources.

• Generate a draft using retrieved approved material.

• Attach citations to important factual/legal/procedural claims.

• Clearly label output as AI-generated and requiring officer review.

• Allow the officer to edit before saving/exporting.

Phase complete when:

• An officer can create a useful first draft.

• The draft remains editable.

• The source material used for the draft is visible.

• The system never presents the draft as a final government decision.

11. Phase 9 — Audit Trail and Administration

Goal: Demonstrate accountability and administrative control.

Build:

• Record login and security events where appropriate.

• Record document upload, processing, approval, rejection and archival actions.

• Record officer queries and document/source views.

• Record draft generation.

• Build an Admin audit-log screen with filters.

• Add document versioning and superseded-document handling.

• Prevent retrieval of superseded/unapproved documents unless explicitly allowed by the retrieval policy.

Phase complete when:

• Admin can reconstruct important system activity.

• Document versions are distinguishable.

• The audit trail links user actions to resources.

12. Phase 10 — Security, Reliability and Demo Hardening

Goal: Make the prototype trustworthy enough for a judging/demo environment.

Build:

• Validate uploads and enforce file limits.

• Sanitize and validate API inputs.

• Protect secrets with environment variables.

• Add rate limiting where appropriate.

• Handle AI-provider failures gracefully.

• Add ingestion retry/error states.

• Add database indexes for common filters and retrieval metadata.

• Test RBAC at API level.

• Test citation correctness.

• Test unsupported questions.

• Add basic logging and monitoring.

Phase complete when:

• The main demo works repeatedly without manual database fixes.

• Security checks happen server-side.

• AI failures produce controlled errors.

• The team can explain known limitations.

13. Phase 11 — Evaluation and Demonstration

Do not evaluate the assistant only by whether the UI looks good. Measure the RAG system.

• **Retrieval quality:** Does the correct source/page appear in the top retrieved evidence?

• **Groundedness:** Are factual claims supported by retrieved passages?

• **Citation accuracy:** Does each citation actually support the associated claim?

• **Abstention:** Does the system avoid inventing answers when evidence is missing?

• **Latency:** How long does ingestion and question answering take?

• **Permission tests:** Can an Officer access any Admin-only endpoint?

• **Document version tests:** Does retrieval use the current approved version?

Prepare 8–15 representative questions

• Straightforward factual/procedural questions.

• Questions requiring multiple documents.

• Questions involving rule numbers/dates.

• A question whose answer is not present.

• A question that should retrieve an older/superseded document but should prefer the current approved version.

• A question that tests department/security filtering.

14. Final Prototype Demo Flow

1. Admin login: show role-based dashboard.

2. Upload: upload a sample government document and enter metadata.

3. Processing: show extraction/OCR → chunking → embedding → processing status.

4. Approval: Admin reviews and publishes the document.

5. Officer login: show that the Officer has no upload/approval controls.

6. Ask: ask a realistic procedural question.

7. Answer: show concise grounded response.

8. Verify: click citation and open the exact source page/snippet.

9. Challenge: ask something absent from the knowledge base and demonstrate refusal/insufficient evidence.

10. Draft: generate a source-backed office-note draft.

11. Audit: show the Admin audit trail for the activity.

15. Future Implementations — After the Prototype

These should be treated as future/production extensions, not requirements for the first build.

Security and government deployment

• Deployment on approved government/on-premise infrastructure.

• SSO/enterprise identity integration and MFA.

• Network segmentation, private service-to-service communication and stronger secrets management.

• Encryption at rest and in transit.

• Security assessments, vulnerability scanning and penetration testing.

• Fine-grained document-level and department-level access policies.

Better retrieval

• Hybrid retrieval combining vector and keyword search.

• Cross-encoder/reranker for better evidence selection.

• Query rewriting and decomposition for complex questions.

• Parent-child or hierarchical retrieval for long regulations.

• Knowledge graph/entity linking for rules, departments, people, dates and document relationships.

• Temporal/version-aware retrieval.

Document intelligence

• Better OCR for poor scans and tables.

• Table extraction.

• Layout-aware document parsing.

• Automatic metadata extraction.

• Document classification and duplicate detection.

• Change detection between document versions.

Bilingual and regional support

• English and Assamese support.

• Multilingual embeddings and retrieval evaluation.

• Bilingual source display.

• Translation with source preservation.

• Careful handling of government terminology and names.

AI reliability

• Formal RAG evaluation datasets.

• Automated citation verification.

• Claim-level grounding checks.

• Confidence/evidence indicators without presenting them as guarantees.

• Model fallback strategies.

• Small/on-premise language models where data policy requires them.

Workflow and integrations

• Integration with e-Office or other approved systems where permitted.

• Email/notification workflows.

• Approval routing.

• Document signing integration.

• Export to official templates.

• API integration with departmental knowledge systems.

Analytics and governance

• Usage analytics for administrators.

• Frequently asked procedural topics.

• Unanswered-question reports to identify missing knowledge.

• Knowledge-base freshness dashboards.

• Document expiry/review reminders.

• Human feedback loops for improving retrieval.

Operational scale

• Asynchronous job queue for large document batches.

• Horizontal scaling of FastAPI workers.

• Caching for repeated queries.

• Observability/tracing.

• Backup and disaster recovery.

• High-availability PostgreSQL/object storage.

16. Recommended Technology Stack

Layer	Recommended choice	Purpose
Frontend	Next.js	Admin and Officer interfaces
Main backend	NestJS	Auth, RBAC, users, documents, approvals, audit, API
AI service	FastAPI / Python	OCR, chunking, embeddings, retrieval, LLM
Database	PostgreSQL + pgvector	Application data, chunks and vectors
File storage	S3-compatible/object storage	Original PDFs
Embeddings	A suitable embedding model/API	Semantic retrieval
LLM	Approved API or deployable model	Grounded generation
Auth	JWT + httpOnly cookie/session	Authentication

17. Build Order — The Short Version

Build in this exact dependency order:

• Foundation: Next.js + NestJS + FastAPI + PostgreSQL/pgvector.

• Auth: login + JWT/session + ADMIN/OFFICER RBAC.

• Documents: Admin upload + metadata + file storage + status.

• Ingestion: extraction/OCR → page-aware chunking → embeddings → pgvector.

• RAG: query embedding → retrieval → grounded LLM answer.

• Citations: document/page/chunk traceability + source viewer.

• Search: normal document search and filters.

• Summarization + structured extraction.

• Drafting assistant.

• Audit logs + versioning.

• Security/reliability hardening.

• Evaluation + polished 5-minute demo.

Definition of a successful MVP

A successful MVP is not the one with the most AI features. It is the one where an Admin can safely publish approved documents, an Officer can ask a realistic question, the system retrieves the correct evidence, the answer is grounded and cited, the Officer can verify the source, unsupported questions are handled safely, and the complete activity can be audited.